Expectation Functions of Minimum Spanning Trees in Various Applications

The Growth of Expectations of Total Weights of Minimum Spanning Trees as Number of Vertices Increases

CO-AUTHORS
Derek Jin, Daniel Potapo

LATE DAYS
Derek Jin:
No. of late days used on previous psets : 0
No. of late days used after including this pset: 2
Daniel Potapo:
No. of late days used on previous psets : 2
No. of late days used after including this pset: 4

PROJECT DATES
Feb 11, 2026 - Feb 20th, 2026

ABSTRACT - Note to the graders: we are *very* sorry for the length of this report. We really really tried to keep things tight and concise, but it just got out of hand with all the things we felt we had to justify and discuss for a complete report. We have summarized our main results in this abstract, but once again: very very sorry! We did *not* mean for it to get this long.

We implemented code for Kruskal's Minimum Spanning Tree (MST) algorithm and tested it across the five dimensions: dimension 0, 1, 2, 3, and 4. At the beginning of the "RESULTS" section, we included our table listing the average tree size for several values of n and our description of our guess for the function f(n).

- For dimension 0, the complete graph on n vertices, we found an empirical expected total MST weight of $f(n) \approx 1.1968$ as a function of n for constant complexity.
- For dimension 1, the hypercube, we found an empirical expected total MST weight of f(n) = $0.4338\frac{n}{\log n}$ for $O(\frac{n}{\log n})$ complexity.
- For dimensions 2-4, we found empirical expectations of $0.6503n^{1/2}$, $0.6546n^{2/3}$, and $0.6952n^{3/4}$ respectively (for $O(n^{(d-1)/d})$ complexity if d = dimension)

To help our model run for higher values of n, we used pruning thresholds (cutoffs above which we discard edges associated with those weights due to their vanishingly-low probability of appearing in our MST). Our pruning threshold functions, denoted T(dim; n), are as follows:

$$T(0;\ n)\ = 2.622 * \log(n)/n$$

$T(1;\ n)\ =$ NO Pruning for Dim 1. Hypercube is already sparse. See "Pruning Math Analysis" section for a deeper discussion.

$$T(2;\ n)\ = 1.429 * (\log(n)/n)^{(1/2)}$$

$$T(3;\ n)\ = 1.603 * (\log(n)/n)^{(1/3)}$$

$$T(4;\ n)\ = 1.549 * (\log(n)/n)^{(1/4)}$$

We discuss pruning in more depth later in our report (including how exactly we implement them and why they don't affect correctness).

INTRODUCTION

The goal of this project was to design an algorithm to simulate various cases of Minimum Spanning Tree (MST) problems and obtain the expected (average) weight of the MSTs for various values of n (number of vertices) for each case. As always, we prove correctness and perform runtime analyses for each combination of case and n. The first phase of our process involved achieving a "Version 1" implementation for our algorithm, which was correct but crude and unoptimized. Using our first phase, we ran a series of mini-experiments to obtain "pruning functions," which we used to optimize our algorithm for higher values of n and obtain our "Version 2" code. <u>Only our final, Version 2 code is submitted,</u> but we note that our Version 1 code helped us construct the "pruning thresholds."

For each implementation in our report, we prove correctness and analyze runtime as required. In our discussion section, we discuss some of the more conceptual and constructive aspects of this project, such as:

- Our rationale behind our choice of algorithm (why Kruskal's over Prim's)
- Why the constant complexity observed in the dimension 0 case was surprising (and an intuitive explanation for why it actually made sense)
- Mathematical derivations justifying each of the observed growth rates
- Experiences with the random number generator.

ALGORITHM

In this section, we explain the implementation we used in our Version 2 code (the optimized version we submitted, with pruning). We justify correctness and analyze runtime. We also use data on the maximum edge weights in specific cases of dimension and n-values along with mathematical analysis to support the safeness of our pruning thresholds. Although this is technically part of correctness, we will include it as its own section for clarity and defer our justifications of the pruning thresholds to that section.

Our runtime analysis (and correctness arguments) will rely on theory and logical arguments, not data. *But,* we do include a column in our data table in the results section dedicated that tracks our runtime analysis. We do this to support our runtime analysis and will remark on that later.

General Correctness Claims:

Lemma 1: If a graph has a minimum spanning tree, is pruned using a threshold t, but is still connected after pruning, there is a minimum spanning tree that has the same weight as the minimum spanning tree in the original graph

Pf: Let G = (V, E, w) be a weighted graph with a MST T. Fix some threshold $0 \leq t \leq 1$ and let G_t be the graph obtained after pruning all edges of weight strictly greater than t.

We first prove the following claim:

Claim: Suppose we have chosen some t such that G_t is still connected. Then, no edge of T has weight greater than t.

Pf: Suppose for the sake of contradiction that T contains some edge e with w(e)>t. Since T is a minimum spanning tree, T is acyclic, so if we remove any edge e from T, T splits into two connected components (call them T_1 and T_2). Since G_t is connected, we know there must be some edge f in G_t with one endpoint in T_1 and the other in T_2. By construction, this weight must satisfy $w(f) \leq t \leq w(e)$. But then, the tree T' = T - {e} ∪ {f} is a MST with lower weight than T, so we have a contradiction.

Therefore, no edge of T has weight greater than t. It follows then that our pruning did not remove any weights in the MST T in G, so that same MST still exists in G_t, and since a spanning tree need only be connected, acyclic, and contain all vertices in the graph, T is also a spanning tree of G_t. And since we only removed edges and did not add or increase any weights, the total weight of T in G_t will still be a minimum amongst the spanning trees of G_t. Therefore, if we prune weights above a threshold t and the graph is still connected, we preserve Minimum Spanning Trees.

Note/Corollary to Lemma 1: Thanks to Lemma 1, it means that if we can find a threshold function T(n) that provides a "probabilistic safe upper bound" on the distribution of the random variable that crystalizes to the maximum weight in the minimum spanning tree, we can optimize our algorithm runtime by first pruning the weights before running Kruskal's. This bound is "probabilistically safe" in the sense that the probability that the maximum weight of the minimum spanning tree is below our threshold function at any given large value of n is very close to 1. This holds because if our pruning for a given weight t does not remove any weights in the MST on G, the resulting graph G_t will still contain the MST, will still be connected, and will therefore still find the correct MST.

Dimension 0 - Complete graph on n vertices

- **Implementation:**
    - **Graph construction:** For a given input n, the graph structure includes n vertices, but since the graph is complete and each of the n(n-1)/2 edges are generated i.i.d.

Unif(0, 1), we just randomly generated n(n-1)/2 random weights for each undirected edge in the adjacency list. Since the graph is undirected, our algorithm was able to get all n(n-1)/2 edges by only considering edges (i, j) where vertex i was less than j. This is discussed in the correctness section down below.

- **Pruning modification to Kruskal's:** Our pruning methodology for dim 0 is as follows:
  - We use the pruning function $T(0;\ n) = 2.622 * \log(n)/n$, which is modeled after our empirical data and analysis (see pruning section)
  - In our Kruskal's implementation, we first check if any weights exceed T(0; n) for a given n. If so, we discard those edges
  - Then, we implement Kruskal's (see next section)
- **Kruskal's implementation:** Run Kruskal and output sum of the weights of a given MST per simulated trial
  - **Kruskal logic:** We implement Kruskal's, which works by sorting the edges in increasing order and greedily choosing the edge with the smallest weight that does not create a cycle
  - **Functions used within Kruskal's (will be relevant for runtime)**
    - **Makeset:** We implement makeset which creates a new set containing a given element x
    - **Sorting edges:** We use the built-in std sort algorithm
    - **Union:** We implement union which replaces two sets with their union
    - **Find:** We implement find, which returns the name of the set containing the input element x
- **Summing and Averaging the weights:**
  - We run k trials of our Kruskal + Pruning Modification algorithm above, and in each trial, we take our MST and iterate through the vector weights and sum them up
  - Then, across the k trials, we take the average of the total MST weights calculated per trial which is what we output (along with the numpoints, numtrials, and dimension to match the output asked for in the PDF)

- **Correctness:**
  - **Graph construction correctness:** Our graph construction in the dimension=0 case is correct because our construction is complete and each of the n(n-1)/2 edges are generated i.i.d. Unif(0, 1).
    - First, we note that taking all edges of the form (i, j) where i is strictly less than j is correct because it gets us all undirected edges on a complete n vertex graph, which are represented by ordered pairs of vertices. Therefore, considering both (i, j) and (j, i) would be unnecessary and

edges (i, i) that go to and from the same vertex would not be included in our graph.
        - Also, a note about the i.i.d. implementation: in the given dimension 0 construction, the edge weights are independent. This will not necessarily be the case when we look at the cases for dimension 2-4 for example, because then, the vertex generation and subsequent Euclidean distances will influence the weights and potentially make them correlated. However, in the dimension 0 case of n vertices in a complete graph, where edges generated i.i.d. uniform, our implementation is safe.
    - **Pruning Modification correctness:**
        - The correctness of our pruning modification follows from our proof of Lemma 1 and our theoretical and empirical in finding safe pruning thresholds (see Pruning Section)
        - We mostly defer to those sections, but we restate the core logic which is that by removing weights above our safe pruning threshold, we remove edges with weights that are so large that it is highly unlikely (probability approx. 0) that they will end up in the final MST
        - Or, in terms of Lemma 1, we find a safe pruning threshold at which the probability of our partially flooded graph G_t remaining connected (per Lemma 1 a partially-flooded graph that is connected preserves the right MST) is very close to 1 (hence correctness)
    - **Kruskal's correctness:**
        - Besides our pruning implementation, our Kruskal implementation follows closely to the one given in section notes and lecture
        - The algorithmic correctness of our UnionFind data structure and the functions MakeSet and UnionFind also follow from our explorations of why Kruskal's works (from lecture)
        - And, since we know that the Kruskal algorithm correctly returns the MST, this gives us what we need for each trial, which is a calculation of the total weight of a simulated MST on our given graph construction
        - Per trial, we calculate the total weight of a simulated MST and then send it to the final part of our algorithm "Summing and Averaging the weights" as required.
    - **Summing and Averaging the weights correctness:**
        - Correctness follows from the correctness of the averaging algorithm ($AVG\ per\ Trial\ = \frac{SUM}{\#Trials}$).
- **(Amortized) Runtime Analysis:** For this case (dim 0) as well as dimension 2-4, pruning will be factored into our runtime analysis, so similarly to how our correctness proofs for pruning rely on convergence to probability 1 of correct output, our runtime analyses in the dim 0 and dim 2-4 cases will rely on amortized runtime analysis. For completeness,

we will also give the runtime we would have gotten without pruning to show the effect of pruning on the asymptotic runtime

- **Without pruning:** Our three main contributors to runtime are the graph construction, sorting edges, and the union-find operations
  - First, we iterate through $O(n^2)$ vertices to sort our graphs to begin with so that immediately gets us to at least $O(n^2)$ runtime
  - Before pruning, the number of edges was $m = n(n-1)/2 = O(n^2)$. Our code uses the built-in std sort function, which has runtime $O(m \log m)$. So sorting edges takes $O(m \log m) = O(n^2 \log n^2) = O(n^2 \log n)$
  - We can use from lecture 6 that no sequence of operations takes more than $O((m+n) \log * n)$, which gives us approximately $O((m+n) \log n)$ amortized. This gives us $O(n^2 \log n)$ (which dominates the $O(n^2)$ from graph construction)
  - Thus, without pruning, our runtime is $O(n^2 \log n)$
- **With pruning:**
  - We still iterate through $O(n^2)$ vertices to sort our graphs to begin with so that immediately gets us to at least $O(n^2)$ runtime
  - After pruning by the function $2.622 * \log(n)/n$ (note that the "$\log(n)/n$" is what really matters asymptotically) the expected number of surviving edges is $m * T(0; n) = O(n^2 * (\log n / n)) = O(n \log n)$.
  - Therefore, m is now $O(n \log n)$ after pruning
  - Again, we look at both our sorting runtime and union-find runtime
    - Sorting is $O(m \log m) = O(n \log n \log (n \log n)) = O(n \log n [\log n + \log \log n])$. After expanding and taking the terms that dominate asymptotically, we get $O(n \log^2 n)$
    - We can use from lecture 6 that no sequence of operations takes more than $O((m+n) \log * n)$, which gives us approximately $O((m+n) \log n)$ amortized. This gives us $O(n \log^2 n)$
  - We note that $O(n \log^2 n)$ is a much better asymptotic runtime then $O(n^2 \log n)$, thus justifying our decision to do pruning
  - However, we are not done. Now, the $O(n^2)$ runtime back when we iterated to construct the graph dominates. We note that n dominates $\log^2 n$ which we can show by taking logs of both and comparing $O(\log n)$ to $O(\log \log n)$. Therefore, our runtime is **$O(n^2)$.**

Dimension 1 - Hypercube

- **Implementation:**
  - **Graph construction:**
    - For a given input n, we construct our hypercube by using iterating through pairs of vertices (with one iterate i always less than the other one j so that

we get all ordered pairs without overcounting), checking if j - i is a power of 2, and if so, adding the edge (randomly generated unif(0, 1)) to our adjacency list

- **(Direct) Kruskal's implementation:** Run Kruskal and output sum of the weights of a given MST per simulated trial
  - **Kruskal logic:** We implement Kruskal's, which works by sorting the edges in increasing order and greedily choosing the edge with the smallest weight that does not create a cycle
  - **Functions used within Kruskal's (will be relevant for runtime)**
    - **Makeset:** We implement makeset which creates a new set containing a given element x
    - **Sorting edges:** We use the built-in std::sort algorithm
    - **Union:** We implement union which replaces two sets with their union
    - **Find:** We implement find, which returns the name of the set containing the input element x
- **Summing and Averaging the weights:**
  - We run k trials of our Kruskal + Pruning Modification algorithm above, and in each trial, we take our MST and iterate through the vector weights and sum them up
  - Then, across the k trials, we take the average of the total MST weights calculated per trial, which is what we output (along with the numpoints, numtrials, and dimension to match the output asked for in the PDF)

- **Correctness:**
  - **Graph construction correctness:** The correctness of our graph construction follows from the given definition of the hypercube for the dimension 1 case in the ProgSet PDF.
  - Like in Dimension 0, we only compare the ordered pairs and we do that by comparing/iterating through edges (i, j) where i < j
  - **Kruskal's correctness:**
    - We don't do pruning for dimension 1, so our Kruskal implementation follows closely to the one given in section notes and lecture
    - The algorithmic correctness of our UnionFind data structure and the functions MakeSet and UnionFind also follow from our explorations of why Kruskal's works (from lecture)
    - And, since we know that the Kruskal algorithm correctly returns the MST, this gives us what we need for each trial, which is a calculation of the total weight of a simulated MST on our given graph construction

        - Per trial, we calculate the total weight of a simulated MST and then send it to the final part of our algorithm "Summing and Averaging the weights" as required.
    - **Summing and Averaging the weights correctness:**
        - Correctness follows from the correctness of the averaging algorithm ($AVG\ per\ Trial\ = \frac{SUM}{\#Trials}$).
- **Runtime Analysis:** For the dimension 1 case, we do not prune so we can just take the canonical runtime of Kruskal's (but we *will* account for the fact that our graph is already sparse)
    - First, we iterate through $O(n^2)$ vertices to sort our graphs to begin with so that immediately gets us to at least $O(n^2)$ runtime
    - As noted previously, the graph in the hypercube case is sparse because each vertex has approximately $\Theta(\log n)$ neighbors, so the number of edges is approximately m = $\Theta(n \log\ n)$ up to scaling constants
    - Besides graph construction, our two main contributors to runtime are sorting edges and the union-find operations
        - Our code uses the built-in std sort function, which has runtime O(m log m) = O((n log n) log (n log n)). We can express "(n log n) log (n log n)" as "n log n * [log n + log log n]" and then expanding gets $O(n \log^2 n)$ runtime
        - For our union-find operations, we can use from lecture 6 that no sequence of operations takes more than O((m+n) log * n), which gives us approximately O((m+n) log n) amortized. This gives us $O(n \log^2 n + n \log n) = O(n \log^2 n)$
    - Therefore, our runtime for Kruskal's in the dimension 1 case is $O(n \log^2 n)$ (amortized)
    - BUT our $O(n^2)$ runtime from the graph construction dominates, so overall runtime is **$O(n^2)$.**

Dimension 2-4 - Unit Square, Unit Cube, and Unit Hypercube

- **Implementation:**
    - **Graph construction:** For dimensions 2-4, we construct our graph by randomly generating points in the d-th dimension unit cube (formally a square for dimension 2, cube for dimension 3, hypercube for dimension 4). Then, we calculate the Euclidean distance using the definition: i.e., for dimension d ∈ {2, 3, 4}, we take $d(x,\ y)\ =\ (\sum_{i=1}^{d} |y_i - x_i|^2)^{1/2}$ and for each vertex we store this euclidean distance as our weight in the adjacency list
    - **Pruning modification to Kruskal's:** Our pruning methodology (which generalizes across dimensions 2-4 as well) is as follows:

        - For dimensions 2, 3, and 4 respectively, we use the pruning functions

$$T(2;\ n) = 1.429 * (\log(n)/n)^{(1/2)}$$
$$T(3;\ n) = 1.603 * (\log(n)/n)^{(1/3)}$$
$$T(4;\ n) = 1.549 * (\log(n)/n)^{(1/4)}$$

          Which is modeled after our empirical data and analysis (see pruning section)
        - In our Kruskal's implementation, for each dimension d, we first check if any weights exceed T(d; n) for a given n. If so, we discard those edges
        - Then, we implement Kruskal's (see next section)
    - **Kruskal's implementation:** Run Kruskal and output sum of the weights of a given MST per simulated trial
        - **Kruskal logic:** We implement Kruskal's, which works by sorting the edges in increasing order and greedily choosing the edge with the smallest weight that does not create a cycle
        - **Functions used within Kruskal's (will be relevant for runtime)**
            - **Makeset:** We implement makeset which creates a new set containing a given element x
            - **Sorting edges:** We use the built-in std sort algorithm
            - **Union:** We implement union which replaces two sets with their union
            - **Find:** We implement find, which returns the name of the set containing the input element x
    - **Summing and Averaging the weights:**
        - We run k trials of our Kruskal + Pruning Modification algorithm above, and in each trial, we take our MST and iterate through the vector weights and sum them up
        - Then, across the k trials, we take the average of the total MST weights calculated per trial which is what we output (along with the numpoints, numtrials, and dimension to match the output asked for in the PDF)
- **Correctness:**
    - **Graph construction correctness:**
        - The correctness of our graph construction follows from the definition of the dimension 2-4, particularly the detail about the weights being represented as the euclidean distances between vertices
        - Like in previous dimensions, we make sure to only calculate weights for ordered pairs so as to avoid overcounting the number of edges (since it's an undirected graph)
    - **Pruning Modification correctness (Same justification as in dim 0):**

        - The correctness of our pruning modification follows from our proof of Lemma 1 and our theoretical and empirical in finding safe pruning thresholds (see Pruning Section)
        - We mostly defer to those sections, but we restate the core logic which is that by removing weights above our safe pruning threshold, we remove edges with weights that are so high that it is highly unlikely (probability approx. 0) that they will end up in the final MST
        - Or, in terms of Lemma 1, we find a safe pruning threshold at which the probability of our partially flooded graph G_t remaining connected (per Lemma 1 a partially-flooded graph that is connected preserves the right MST) is very close to 1 (hence correctness)
    - **Kruskal's correctness:**
        - Besides our pruning implementation, our Kruskal implementation follows closely to the one given in section notes and lecture
        - The algorithmic correctness of our UnionFind data structure and the functions MakeSet and UnionFind also follow from our explorations of why Kruskal's works (from lecture)
        - And, since we know that the Kruskal algorithm correctly returns the MST, this gives us what we need for each trial, which is a calculation of the total weight of a simulated MST on our given graph construction
        - Per trial, we calculate the total weight of a simulated MST and then send it to the final part of our algorithm "Summing and Averaging the weights" as required.
    - **Summing and Averaging the weights correctness (same justification across all dims):** Correctness follows from the correctness of the averaging algorithm ($AVG\ per\ Trial\ = \frac{SUM}{\#Trials}$).
- **(Amortized) Runtime Analysis:** Similar to the dim 0 case, we factor pruning into our runtime analysis. And once again, for completeness, we will also give the runtime we would have gotten without pruning to show the effect of pruning on the asymptotic runtime
    - **Without pruning:**
        - First, we iterate through $O(n^2)$ vertices to sort our graphs to begin with so that immediately gets us to at least $O(n^2)$ runtime
        - Besides graph construction, our two main contributors to runtime are sorting edges and the union-find operations
        - Before pruning, the number of edges was $m = n(n-1)/2 = O(n^2)$. Our code uses the built-in std sort function, which has runtime $O(m \log m)$. So sorting edges takes $O(m \log m) = O(n^2 \log n^2) = O(n^2 \log n)$

        - We can use from lecture 6 that no sequence of operations takes more than O((m+n) log * n), which gives us approximately O((m+n) log n) amortized. This gives us $O(n^2 \log n)$
        - So similar to the dim 0 case, without pruning, our runtime is $O(n^2 \log n)$ (without pruning, Kruskal's dominates the $O(n^2)$ runtime from graph construction)
    - **With pruning:**
        - We still iterate through $O(n^2)$ vertices to sort our graphs to begin with so that immediately gets us to at least $O(n^2)$ runtime
        - Similar to the dim 0 case, pruning helps our asymptotic runtime, particularly by reducing the number of edges we need to run through Kruskal
        - Suppose we fix a vertex v. The vertices that get pruned are then the vertices within the d-dimensional ball of radius T(d; n)
        - As noted previously, the volume of a d-dim ball is proportional to $T(d; n)^d$ so each edge's "probability of being safe" is proportional to $T(d; n)^d$. While the d-dim ball may run into edges, we salvage this by noting that on average, the volume even when cutoff should still give us probability that is on the order of $T(d; n)^d$ asymptotically, but perhaps differing by constants
        - Thus, we can get the expected number of surviving edges per vertex by the function $m_V = O(n * T(d; n)^d)$ which gives us O(n * (log n / n)^(d * 1/d)) = O(log n)
        - Thus, we can estimate the expected *total* number of surviving edges by m = O(n log n).
        - Now, we use this new m value to analyze complexity for both the sorting operation and the union find operations
            - Sorting gives us O(m log m) which we have already worked out previously to be $O(n \log^2 n)$ in the m = O(n log n) case
            - Similarly, the union find operations give us O((m+n) log * n) which gives us approximately $O(n \log^2 n)$ amortized
        - Then, our overall *Kruskal's* runtime is $O(n \log^2 n)$ which once again marks a big improvement asymptotically w.r.t. the unpruned runtime
        - However, our graph construction runtime dominates again to give us overall $\mathbf{O(n^2)}$ runtime

Pruning Threshold Justifications

As alluded to above, the idea behind the pruning method is to set a "safe" threshold for each dimensional case and n-value such that edges with weights above that threshold can be discarded with extremely high confidence that they will not appear in the minimum spanning tree. As such,

the threshold is set to be a little higher than the expectation of the maximum weight in the minimum spanning tree. In Stat 110 terms, we can let the random variable W_{max} be the maximum weight of the minimum spanning tree for any given dimensional case and n value. Then, we wish to set a cutoff that is sufficiently above the mean of the W_{max} distribution such that the W_{max} CDF, evaluated at our threshold, is approximately 1. This guarantees that none of the edges we "prune" will have appeared in our MST, which preserves correctness. By fixing the dimension but letting n vary, we get a pruning threshold function T(n), which allows us to better optimize our runtime and achieve higher n cases.

Pruning Math Analysis:

In this subsection, we heuristically estimate the approximate theoretical (asymptotic) forms of the W_{max} distribution function in each of the dimension cases, and use the result to motivate our curve fit for the empirical data we collected on W_{max}.

First, we handle the Dim 1 case and explain why pruning here would be problematic. Suppose we decided to prune in the Dim 1 case, which is the case of the hypercube defined by $|a - b| = 2^i$ for some i. This graph is not complete. Given n vertices, each vertex i in {1, 2, …, n} has at most O(log n) neighbors. Therefore, the total number of edges is O(n log n), which is substantially less than n^2, so the graph is sparse. Therefore, pruning would not make much sense because the purpose of pruning is to make a dense graph more sparse to better optimize runtime.

For a more empirical demonstration of this, see Pruning Graph 2 down below which corresponds to the dimension 1 case and further illustrates why pruning for the sake of optimization is not ideal in the Dimension 1 case.

Next, for **dimension 0 and 2-4,** we use the graph construction, lemma 1, and probability theory to get a theoretical estimation for the approximate (asymptotic) form of the W_{max} distribution. Since our threshold functions are set to be a "safe" distance above the W_{max} distribution, this should give us a decent guide for what type of curve we should fit our pruning graphs to.

For the **dimension 0 case,** we claim that the form of our pruning functions should be approximately of the form $\frac{\log n}{n}$.

Justification: In lecture 5 notes, we introduced Kruskal's algorithm which is essentially a globally greedy MST algorithm that works by sorting edges in order of increasing weights and taking the lightest edge that does not induce a cycle. We want to estimate the approximate form of the distribution of W_{max}.

Using Lemma 1, we can estimate the asymptotic form of W_{max} using the minimum weight t satisfying that all n vertices in G_t has at least one neighbor. We note that this is a necessary (but not sufficient) condition for proving connectivity as if a given weight t does not satisfy this property (i.e. t is too small that it induces an isolated vertex on G_t), our weight t will have to be

lower than W_{max}. Since having no isolated vertices is a strong partial requirement for connectivity, we make the assumption that the asymptotic form of our given t will be similar to the asymptotic form of the W_{max} distribution.

Since any given vertex has n-1 neighbors on a complete graph, for any given t, the probability that a given neighbor is "drowned out" by the weight t is the probability that its weight exceeds t which is $1 - t$. Across all n-1 neighbors then, the probability that all of the vertices are drowned out is $(1 - t)^{n-1}$. For large n, our threshold t is small because our MST is taken among many more spanning trees. And, for small t, we can approximate $(1 - t)^{n-1} \approx (1 - \frac{1}{N})^{n-1}$ where both N = 1/t and n = $N \cdot \frac{1}{N} \cdot n$ are large, which using the limit definition of e, gives us an approximate probability of an isolated vertex existing for a given weight t as $(1 - \frac{1}{N})^{n-1} \approx e^{-(\frac{1}{N} \cdot n)} = e^{-tn}$. Then, by Boole's, we can crudely bound the probability that there exists some isolated vertex across the n vertices by taking the sum of the probabilities over the n vertices, so the probability that G_t has at least one isolated vertex for a given weight t is approximately less than or equal to ne^{-tn}. Therefore, by solving for t in $ne^{-tn} = 1/2$, we get what is a hopefully decent approximation of the weight t at which the probability of having no isolated vertices is around 50% (and this is what we will take as our approximation for W_{max}). Solving for t, we get t = $-\ln(\frac{1}{2n})/n = ln(2n)/n = O(\log(n)/n)$. Finally, we use this to get our estimation of the asymptotic form of W_{max}. We **note** that this is a very crude approximation and its only purpose is to help us pick a curve-fit form for our pruning thresholds (which will be based on empirical data).

For the **dimension 2-4 case,** our graph structure changes compared to the dimension 0 case. Let dimension = d. We still have a complete graph on n vertices, but now, the weights are taken as the Euclidean distances between respective points, not generated i.i.d. Unif(0, 1). As with the dimension 0 case, Kruskal's algorithm sorts the edges in order of increasing weight, so we can approximate the asymptotic form of the threshold curve by "flooding" the weights in the graph with some threshold t and analyzing the resulting graph.

As before, finding the exact distribution of W_{max} is extremely difficult because it requires showing the precise weight that connectivity hinges on in our partially-flooded graph G_t. Instead, we will again attempt to approximate the asymptotic form by analyzing a strong partial requirement for connectivity, which is that all vertices must have at least one neighbor (or equivalently, no isolated vertices). Our hope is that while no isolated neighbors is not a sufficient condition for connectivity, it is a strong enough partial condition that its asymptotic form correlates very closely with the distribution of W_{max}.

Once again, we will start by fixing some vertex i in our graph. The vertex i will be isolated in a partially-flooded graph G_t if and only if the closed d-th dimensional ball of radius t contains at

least one other vertex j. Assuming i is far enough from the edges and t is small enough such that the d-th dimensional closed ball of radius t does not intersect much with the boundary of the cube, the probability that any given vertex j falls within said ball can be approximated as the area/volume/hyper-volume of said ball. In dimension 2, this means the circle formula πt^2; in dimension 3, this means the sphere formula $\frac{4}{3}\pi t^3$, and in dimension 4, this means the 4d hyper-sphere formula $\frac{1}{2}\pi^2 t^4$. For simplicity, generalizability, and convenience, we will ignore the constant factors in front of the t's and just approximate the probability that a given vertex j does not fall in the closed d-dim ball of radius t around vertex i for a given dimension d as $\approx 1 - t^d$.

Now, considering the probability across all n - 1 other vertices we get the crude approximation P(i isolated) = $(1 - t^d)^{n-1}$ which again for small t can be approximated as P(i isolated) ≈ $e^{-(n-1)t^d} \approx e^{-nt^d}$. Then, using the crude Boole's bound again, we can approximate P(all vertices i isolated) ≈ ne^{-nt^d}. Solving for t = ½, we get the following crude estimator for the asymptotic growth rate of t: $t \approx (-\frac{\ln(\frac{1}{2n})}{n})^{1/d} = (\frac{\ln(2n)}{n})^{1/d}$, so t ≈ O($(\frac{\log(n)}{n})^{1/d}$). Since no isolated vertices is a strong partial requirement for connectivity on a partially flooded graph, our hope is that W_{max} possesses a similar asymptotic distribution to the calculated asymptotics for t.

Therefore, across dimensions 0, 2, 3, and 4, our function form approximations that we will use for our curve fits are

$$T(n) = \frac{\log n}{n} \text{ for dim 1}$$

$$T(n) = (\frac{\log(n)}{n})^{1/2} \text{ for dim 2}$$

$$T(n) = (\frac{\log(n)}{n})^{1/3} \text{ for dim 3}$$

$$T(n) = (\frac{\log(n)}{n})^{1/4} \text{ for dim 4.}$$

We stress that these are only approximations to help guide our pruning implementation.

Pruning Table 1: We support our mathematical analysis with data. For each of the dimensions (0, 1, 2, 3, 4) and n-value combinations, we ran 20 test cases with the corresponding empirical maximum edge weight (i.e., crystallizations of the random variable W_{max}). We recorded the maximum weight obtained across these 20 trials and recorded it in the Maximum W_{max}… column in the table below. Then, we scaled each of our maximum weights up by a factor of 1.5 to give ourselves some extra margin with our thresholds. We recorded these in the pruning threshold point estimates (x1.5) column of our table below.

Note 1: The reason these pruning thresholds should be relatively safe is because our dual methodology of generating 20 weights and taking the maximum and then scaling by 1.5 provides two layers of security, so the probability that W_{max} crystallizes to a value above our set threshold is very low (close to 0). By Lemma 1 above, as long as W_{max} stays below our pruning threshold, our algorithm will return the correct total MST weight.
Note 2: The n-values tested are *different* from the n-values used when analyzing growth functions of expected MST in each dimension case (i.e. the *main part* of the assignment). This is ok because our motivation for analyzing safe pruning thresholds is to optimize runtime so we can get to the higher n-cases later on.

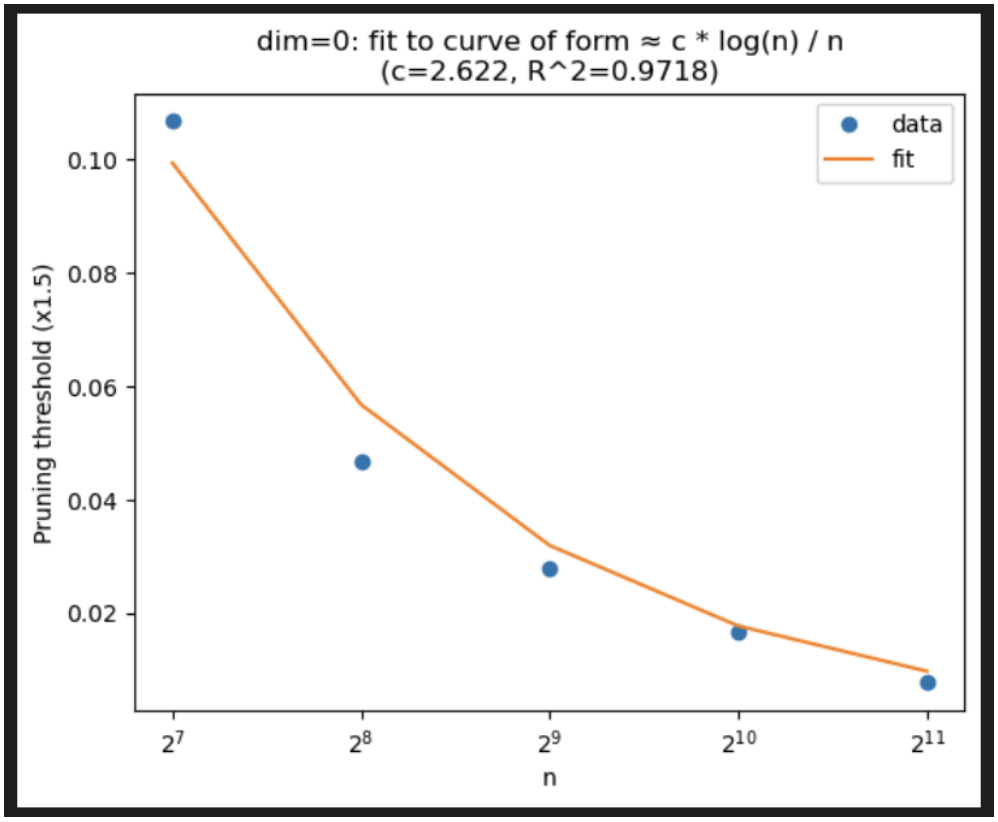
Dimension	n-value	Maximum W_{max} Simulated Over 20 Trials	Pruning Threshold Point estimates (x1.5)
0	128	0.0711887	0.10678305
	256	0.0312372	0.0468558
	512	0.0185376	0.0278064
	1024	0.0110982	0.0166473
	2048	0.00516595	0.007748925
1	128	0.543139	0.8147085
	256	0.489855	0.7347825
	512	0.493578	0.740367
	1024	0.497021	0.7455315
	2048	0.447279	0.6709185
2	128	0.186807	0.2802105
	256	0.162359	0.2435385
	512	0.0881035	0.13215525
	1024	0.0633325	0.09499875
	2048	0.0512979	0.07694685
3	128	0.370116	0.555174
	256	0.28509	0.427635

	512	0.246181	0.3692715
	1024	0.188751	0.2831265
	2048	0.180345	0.2705175
4	128	0.458701	0.6880515
	256	0.416118	0.624177
	512	0.336663	0.5049945
	1024	0.276264	0.414396
	2048	0.24979	0.374685

We note that our pruning threshold point estimates are safe on two fronts:

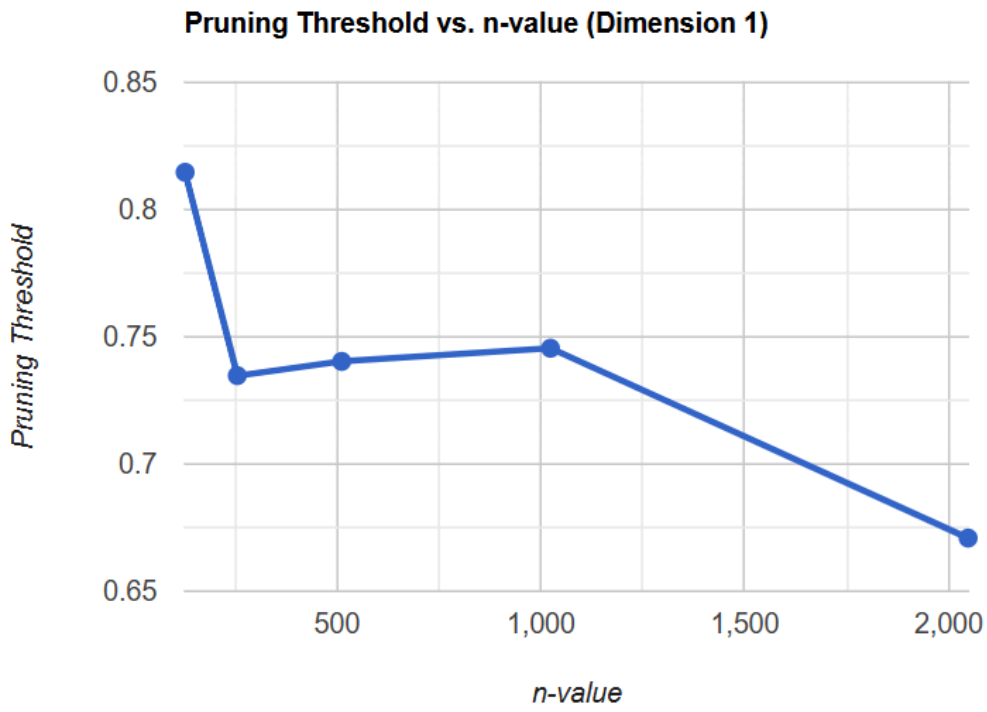
This provides empirical support for the safety of our pruning threshold point estimates. Now we plot our pruning estimates point estimates and fit curves to them to estimate our safe pruning threshold functions.

Pruning Graph 1



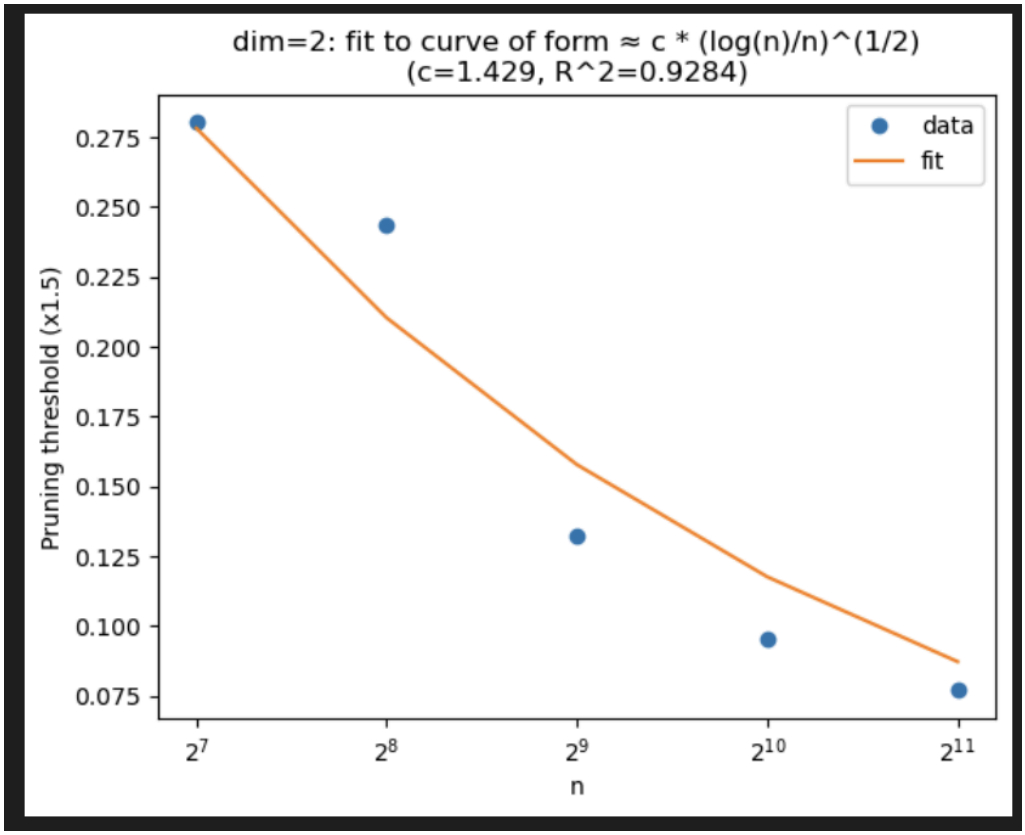
Notes: We fit our plot of five threshold point estimates to the curve of form c * log(n)/n. We got an optimal constant of c = 2.622 with R^2 value of 0.9718. This is decent for our purposes, so we take T(n) = 2.622 * log(n)/n as our pruning function.

Pruning Graph 2



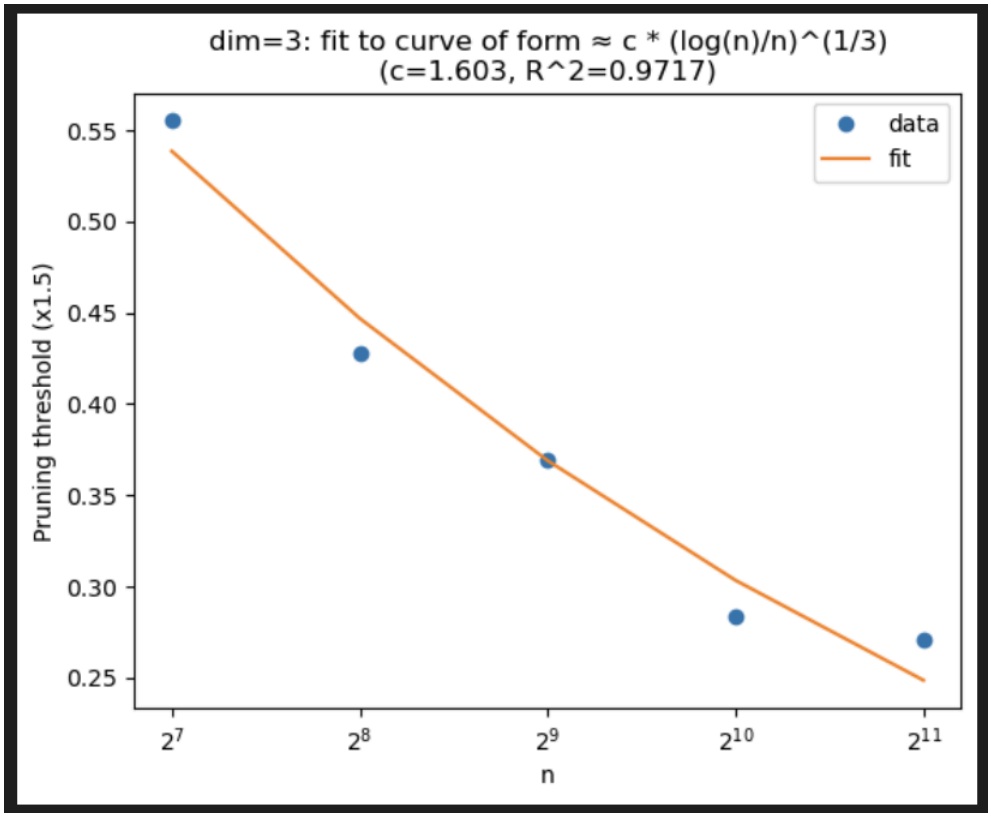
Notes: We *do not* include this graph for the sake of calculating the pruning function. In fact, we are *only including it* to justify our decision *not* to prune for dimension 1. As mentioned above, in dimension 1 (the hypercube), our graph is fairly sparse, so pruning wouldn't really be worth it since much of the runtime optimization gained from pruning comes from pruning dense graphs. We also remark that for a pruning function to be useful on *large n*, it should ideally be monotone decreasing so that the cutoff for deciding which weights we can throw away decays as n increases.

Pruning Graph 3



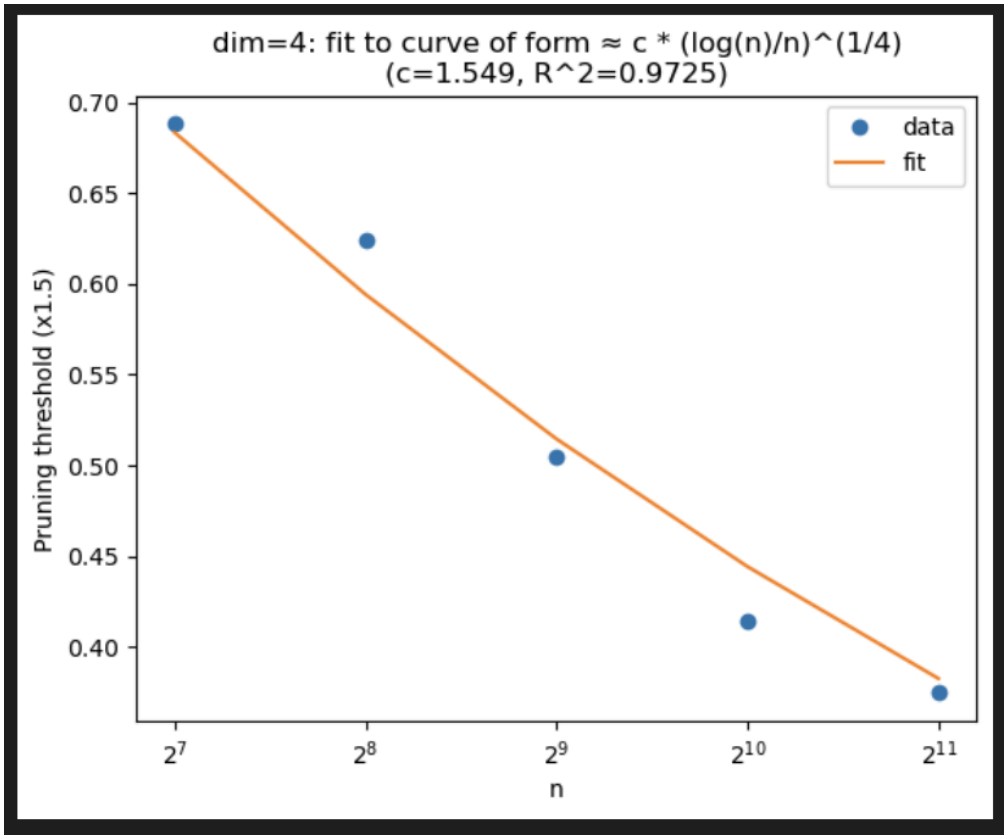
Notes: We fit our plot of five threshold point estimates to the curve of form c * (log(n)/n)^(½). We got an optimal constant of c = 1.429 with R^2 value of 0.9284. This is acceptable for our purposes, so we take T(n) = 1.429 * (log(n)/n)^(½) as our pruning function.

Pruning Graph 4



Notes: We fit our plot of five threshold point estimates to the curve of form c * (log(n)/n)^(⅓). We got an optimal constant of c = 1.603 with R^2 value of 0.9717. This is pretty good for our purposes, so we take T(n) = 1.603 * (log(n)/n)^(⅓) as our pruning function.

Pruning Graph 5



Notes: We fit our plot of five threshold point estimates to the curve of form c * (log(n)/n)^(¼). We got an optimal constant of c = 1.549 with R^2 value of 0.9725. This is pretty good for our purposes, so we take T(n) = 1.549 * (log(n)/n)^(¼). as our pruning function.

RESULTS

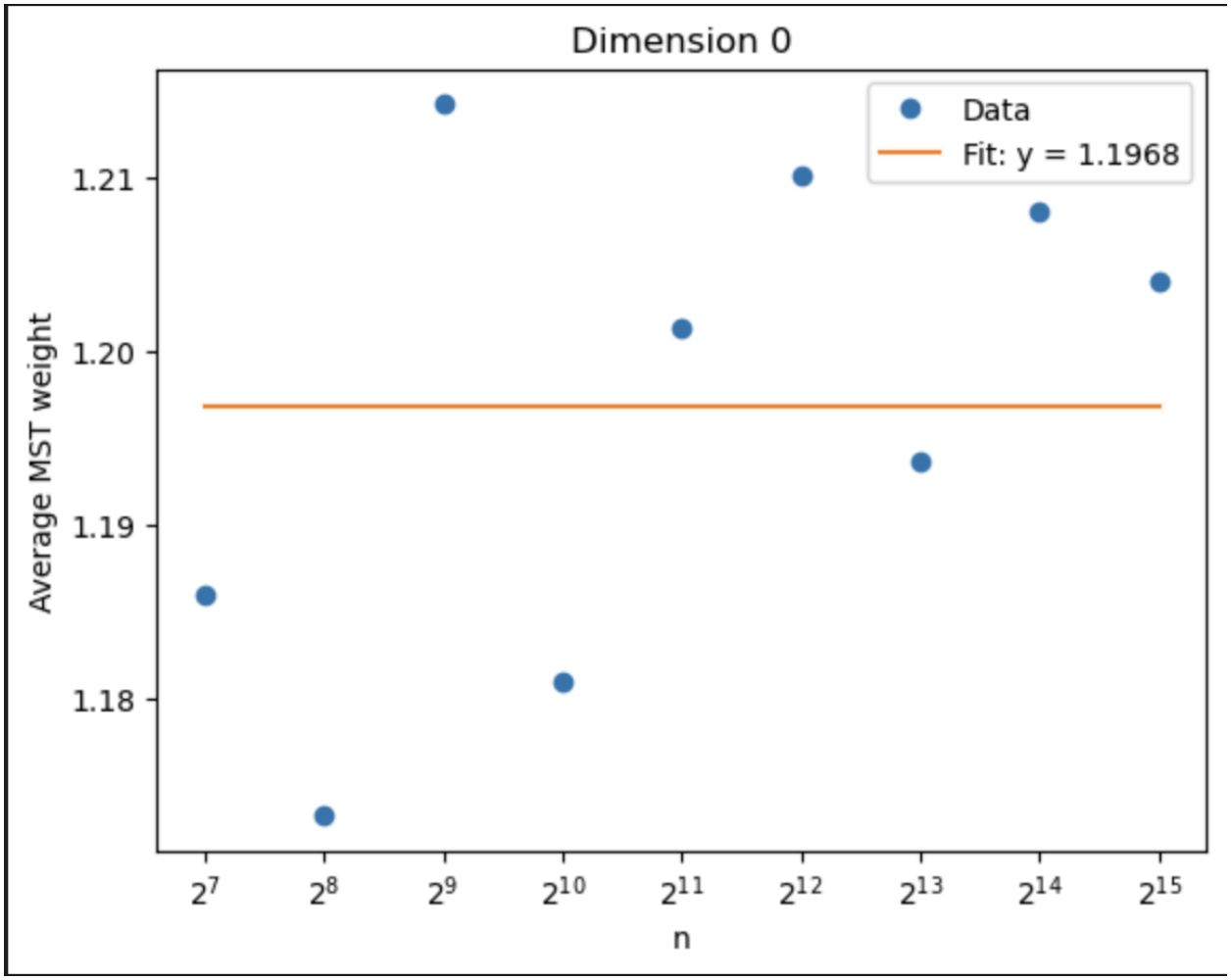
In this section we give our table listing the average tree size for our values of n and our interpretation of the various f(n) functions (with curve fits as empirical support). We have 5 separate tables: one for each dimension. For each n-value for a given dimension, we give the *average* total MST weight taken over 5 random trials of that particular n-value case. We also give the runtime at each of these n-values. Finally, we give estimates of the empirical runtime/growth rate of our algorithm and compare them against our theoretical runtimes from the previous section.

Main Table 1: For Dimension 0: the complete graph on n-vertices case, we list the total MST weight averages taken over 5 trials at each of the n-values. We also list the total runtime for the 5 trials.

n-value	Total MST weight average	Runtime (ms)

128	1.18597	7.0914
256	1.17328	19.3874
512	1.21419	73.9494
1024	1.18098	289.916
2048	1.20129	1,098.95
4096	1.21003	4,356.88
8192	1.19367	24,420.8
16384	1.20799	100,001
32768	1.20398	421,405

Main Graph 1: Dimension 0 case. Lineplot of Average total MST Weights vs. n-values. Fit to constant functions y = c



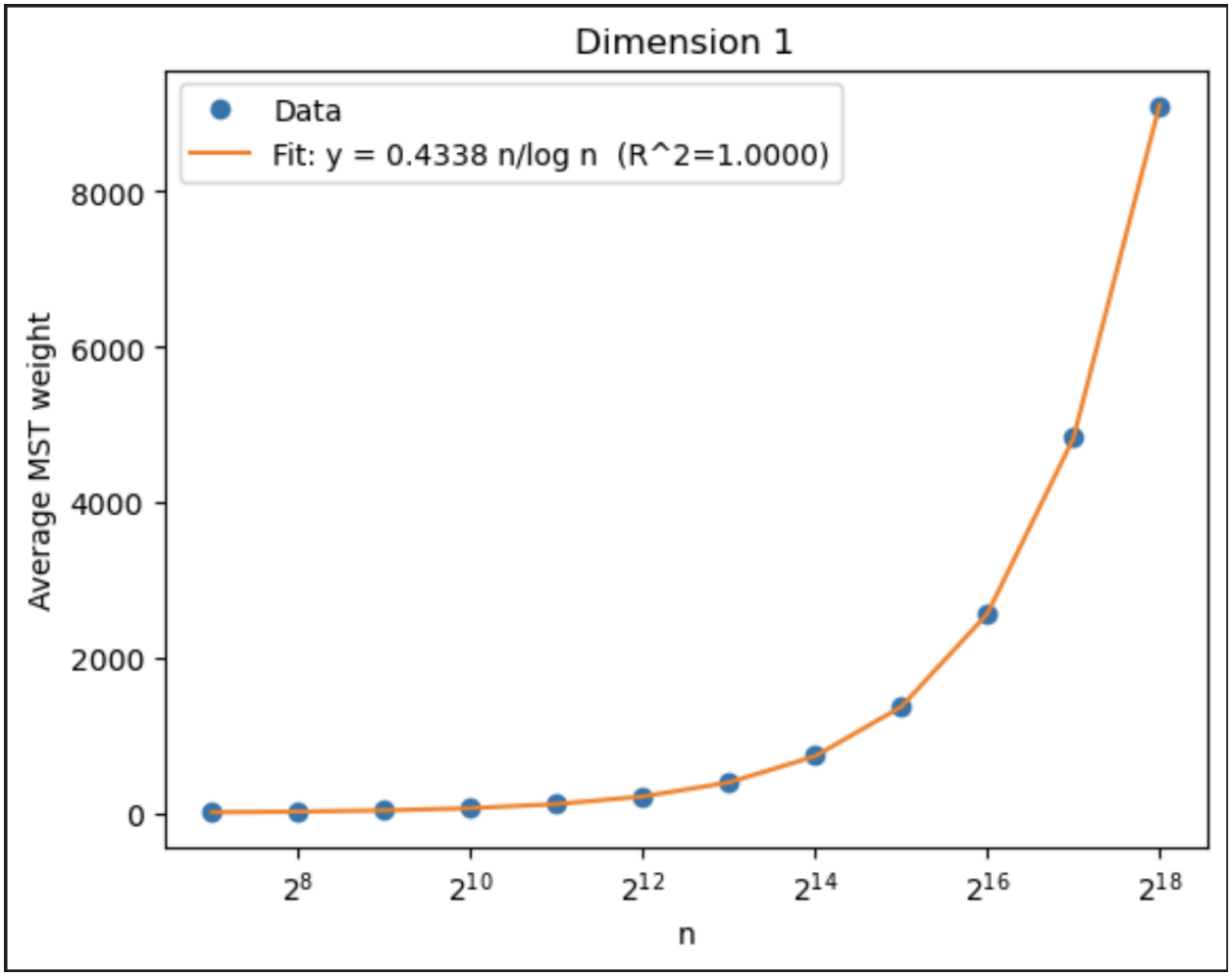
We fit our scatter plot to a constant fit after observing that the pattern seemed roughly constant. We could have also taken an average but plotting a constant fit felt like a good way to justify constant growth. Observe that visually, the residuals in our graph look approximately random. The scale has also been narrowed down significantly for greater visibility between points, but the variance among our points are quite tight: values range between 1.17 and 1.22. Thus, a constant fit seems appropriate.
Therefore, our function in the dimension 0 case appears to be $f(n) \approx 1.1968$ (constant growth).

Note on the Runtime! Looking at the runtime column, notice that as n doubles, the runtime appears to approximately quadruple (this holds better for larger n). This provides some empirical support in favor of our conclusion that overall runtime is $O(n^2)$. We also don't expect to see *exact* quadrupling because $O(n^2)$ is just the asymptotic behavior. So there is room for the quadrupling pattern to vary at the level of n's we're testing!

Main Table 2: For Dimension 1: the hypercube case, we list the total MST weight averages taken over 5 trials at each of the n-values. We also list the total runtime for the 5 trials.

n value	Total MST weight average	Runtime (ms)
128	12.3442	1.1662
256	21.697	1.8936
512	38.1065	5.1316
1024	65.8787	18.4969
2048	120.045	41.4022
4096	218.63	116.603
8192	399.99	350.497
16384	743.849	1,179.32
32768	1,380.4	4,159.88
65536	2,576.09	16,860.6
131072	4,837.88	68,070.1
262144	9,102.06	365,454

Main Graph 2: Dimension 1 case. Lineplot of Average total MST Weights vs. n-values. Fitted to curves of form $c\,\frac{n}{\log n}$. $R^2 = 1.0000$.

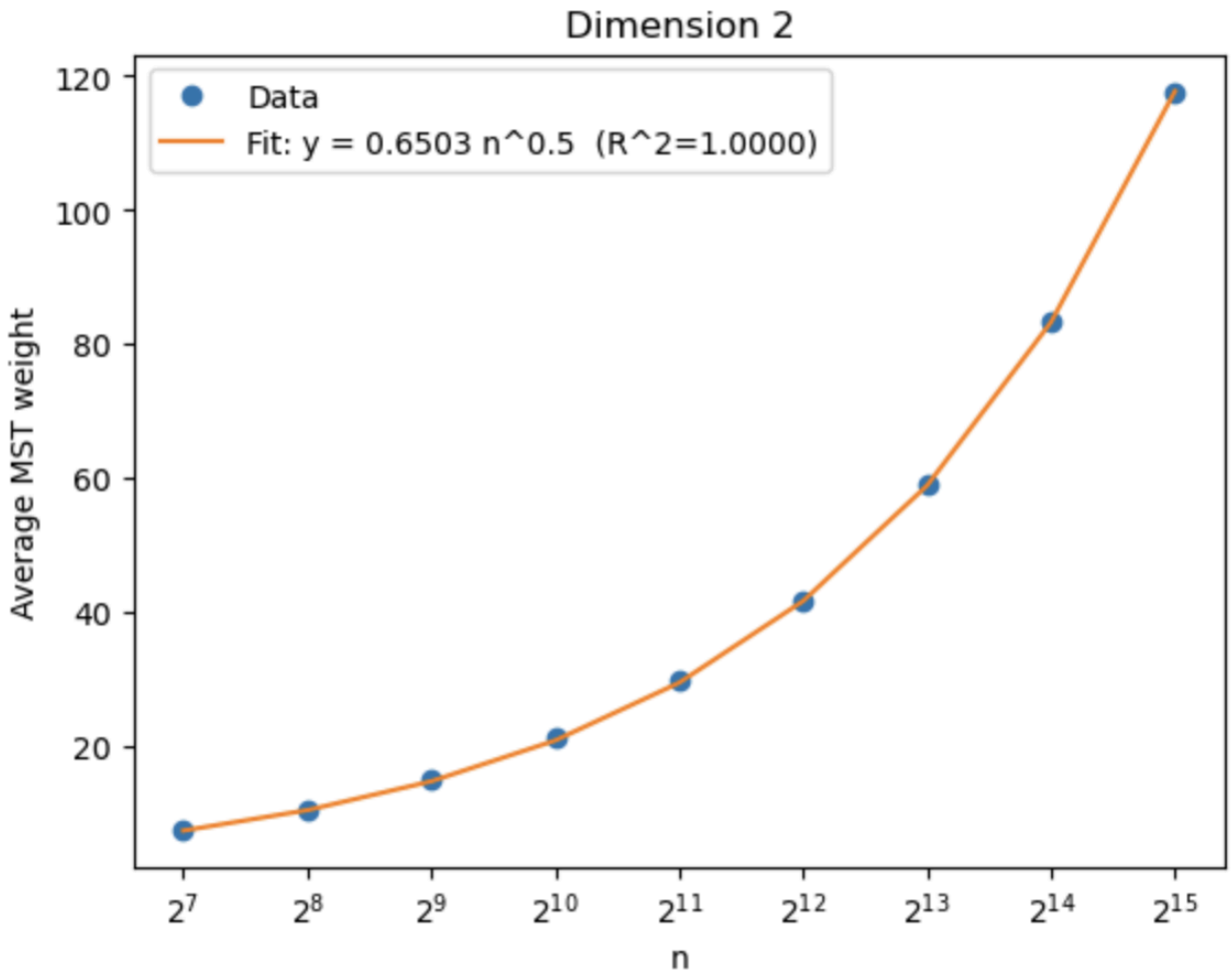


We fit functions of the form $c\,\frac{n}{\log n}$ to our graph and got a fit of $y = 0.4338\frac{n}{\log n}$, where y is the predicted average total MST weight. Our R^2 value was 1.0000 which is very close to 1, suggesting that our approximation was very good. This suggests that the growth of the Average total MST Weights as a function of the n-values in the dimension 1 case is approximately $f(n) = 0.4338\frac{n}{\log n}$ ($O(\frac{n}{\log n})$ growth).

Main Table 3: For Dimension 2: the unit square case, we list the total MST weight averages taken over 5 trials at each of the n-values. We also list the total runtime for the 5 trials.

n value	Total MST weight average	Runtime (ms)
128	7.57431	3.7828
256	10.4979	11.7571
512	14.9129	48.9231
1024	21.1677	167.35
2048	29.711	612.611
4096	41.7717	2,338.16
8192	58.9371	9,197.01
16384	83.2619	37,111.7
32768	117.44	196,165

Graph 3: Dimension 2 case. Lineplot of Average total MST Weights vs. n-values. Fitted to curves of the form $cn^{1/2}$. $R^2 = 1.0000$.

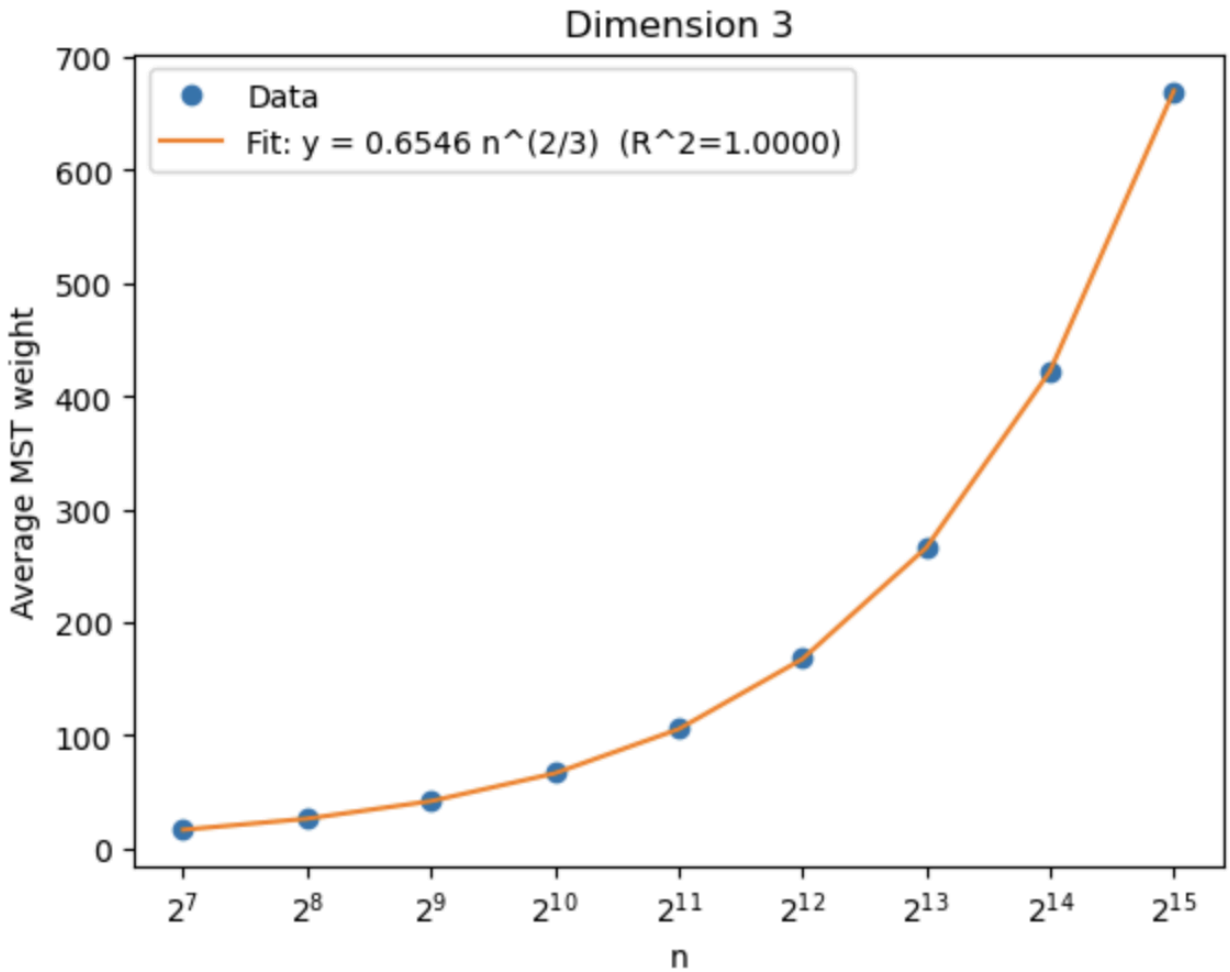


We fit functions of the form $cn^{1/2}$ to our graph and got a fit of $y = 0.6503n^{1/2}$, where y is the predicted average total MST weight. Our R^2 value was 1.0000 which suggests that our approximation was *very* good. This suggests that the growth of the Average total MST Weights as a function of the n-values in the dimension 2 case is approximately $f(n) = 0.6503n^{1/2}$ ($O(n^{1/2})$ growth).

Main Table 4: For Dimension 3: the unit cube case, we list the total MST weight averages taken over 5 trials at each of the n-values. We also list the total runtime for the 5 trials.

n value	Total MST weight average	Runtime (ms)
128	17.9059	6.455
256	27.8796	50.9633
512	42.778	128.908
1024	67.9393	462.269
2048	107.57	1,663.07
4096	169.478	5,696.77
8192	267.161	21,317.1
16384	422.4	76,587.9
32768	668.662	348,152

Main Graph 4: Dimension 3 case. Lineplot of Average total MST Weights vs. n-values. Fitted to curves of the form $cn^{2/3}$. $R^2 = 1.0000$.

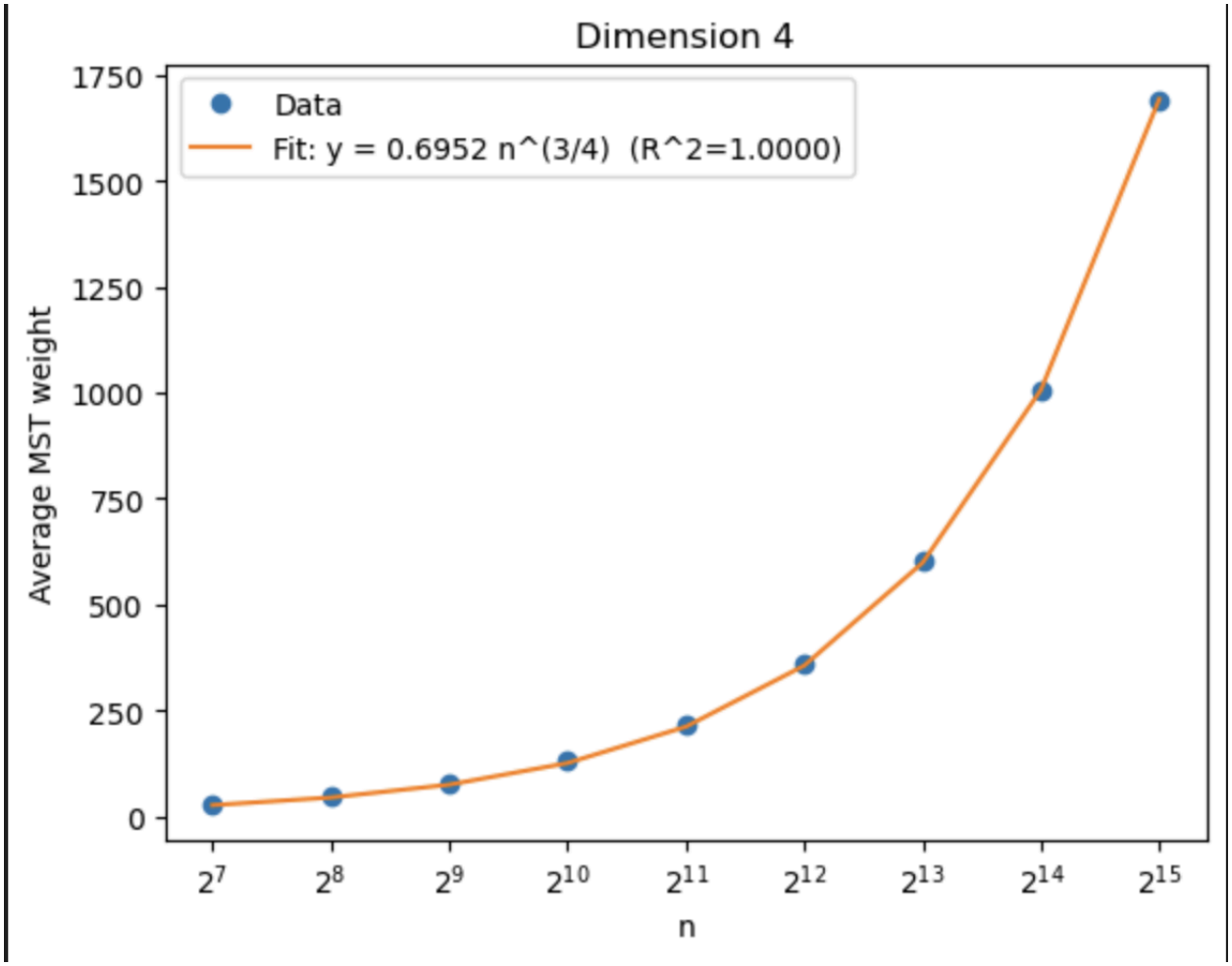


We fit functions of the form $cn^{2/3}$ to our graph and got a fit of $y = 0.6546n^{2/3}$, where y is the predicted average total MST weight. Our R^2 value was 1.0000 which suggests that our approximation was very good. This suggests that the growth of the Average total MST Weights as a function of the n-values in the dimension 3 case is approximately
$f(n) = 0.6546n^{2/3}$ ($O(n^{2/3})$ growth).

Main Table 5: For Dimension 4: the unit cube case, we list the total MST weight averages taken over 5 trials at each of the n-values. We also list the total runtime for the 5 trials.

n value	Total MST weight average	Runtime (ms)
128	29.0208	11.6301
256	47.3534	52.0464
512	77.6401	156.6
1024	129.964	505.996
2048	216.376	1,878.77
4096	360.88	7,530.88
8192	603.013	29,298.4
16384	1,007.18	111,847
32768	1,688.99	430,735

Main Graph 5: Dimension 4 case. Lineplot of Average total MST Weights vs. n-values. Fitted to curves of the form $cn^{3/4}$. $R^2 = 1.0000$.



We fit functions of the form $cn^{3/4}$ to our graph and got a fit of $y = 0.6952n^{3/4}$, where y is the predicted average total total MST weight. Our R^2 value was 1.0000 suggesting that our approximation was very good. This suggests that the growth of the Average total MST Weights as a function of the n-values in the dimension 4 case is approximately
$f(n) = 0.6952n^{3/4}$ ($O(n^{3/4})$ growth).

DISCUSSION

- **Our rationale behind our choice of algorithm (why Kruskal's over Prim's)**

We initially leaned towards Prim's over Kruskal's because Prim's performs better in dense graphs (more edges), and four out of five dimensions involve complete graphs. However, we changed our minds as the project went on because we realized that while Prim's works better before pruning, after pruning (removing edges) our graph becomes sparse. Kruskal's performs better on sparser graphs, so we decided to go with Kruskal's.

- **Why the constant complexity observed in the dimension 0 case was surprising (and an intuitive explanation for why it actually made sense)**

Our data suggested constant complexity in the dimension 0 case (n vertices, dense graph). This was surprising because intuitively, as the number of vertices increase, the number of edges in the MSTs increase, so one might expect the average MST-weight to increase as well (i.e. the notion that more edges per MST implies greater total sum).

However, constant complexity actually is not that surprising when we consider the totality of what happens when n increases. Although the number of edges in each MST increases, the number of possible spanning trees that have the potential to be MSTs increases as well. From Stat 110 order statistics, this means that the MSTs selected will on average have lower weights per edge as the number of vertices increases. So intuitively, we can conceptualize the constant complexity in the dimension 0 case as these two forces "balancing out."

- **Why the fact that complexity generally increased in the other dimensions made sense (dim 1, 2, 3, 4)**

This is kind of hinted at in the previous section, but the naive/intuitive explanation for why this seems reasonable is because as the number of vertices increase, the number of edges in the MSTs increase, so we *should* expect the average MST-weight to increase. Now similar to the dim 0 case, the *average weight* (for the individual edge weights) within the MSTs should *decrease* as n increases because we are selectively choosing lower and lower edges on average. But unlike in the dim 0 case, these effects do not balance out - the increasing number of edges dominates, and results in the expected MST total weight increasing as n increases (in the dim 1, 2, 3, 4 case).

- **Heuristic Mathematical derivations justifying the observed complexity asymptotics for each dimension**

Dimension 0 - Complete Graph on n vertices
Claim: Expected total MST weight complexity is O(1) (constant).
Heuristical Justification:

- The number of edges m is n choose 2 which is $\Theta(n^2)$
- Each edge is i.i.d. Unif(0, 1)
- By order statistics, the minimum is distributed Beta(1, m) and has expectation $\frac{1}{1+m}$
- Additionally, the k-th smallest edge weight post sorting is distributed Beta(k, m-k+1) also by order statistics, and has expectation $\frac{k}{m+1}$
- MST on graph with n vertices always has n-1 edges (property of a tree: |E|=|V|-1)

- Heuristically, since we are selecting for a *Minimum* spanning tree, these edge weights are going to be selected overwhelmingly from the very lower end of the order statistics distribution (in fact, by the correctness of Kruskal's, the absolute smallest weight will always be in our MST)
- Thus, we can heuristically approximate the complexity as $\sum_{k=1}^{n-1} \frac{k}{m+1} = \frac{\Theta(n^2)}{\Theta(n^2)} = \Theta(1)$.
    - This calculation assumes that the MST happens to contain the (n-1)-lowest weights which certainly is not realistic, but for a heuristic asymptotic complexity approximation, should work because while the n-1 edges chosen for our MST won't be the absolute lowest set of n-1 edges, they will still be significantly weighted towards the lower end, to a degree that our complexity will actually be constant

Dimension 1 - Hypercube

Claim: Expected total MST weight complexity is O($\frac{n}{\log n}$).

Heuristical Justification:

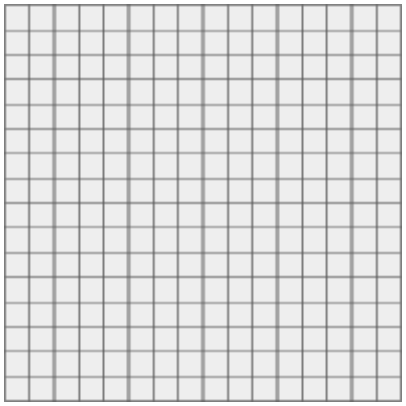
- As noted above, the number of edges in the hypercube graph construction is asymptotically m = $\Theta(n \log\ n)$
- We also note structural similarities between the dim 0 case and the dim 1 case: notably, like the dim 0 case and unlike the dim 2-4 case, the weights are i.i.d. Unif(0, 1) because knowing there's the weight between (i, j) does not tell us anything about the values of the other weights (whereas knowing the weight between (i, j) in the Euclidean cases *would* tell us something, namely how far apart the i and j vertices are)
- The key difference here is the graph is sparser, so instead of each point having approximately n neighbors, each point has approximately $\log n$ neighbors
- Using order statistics again, the minimum weight will be distributed Beta(1, m) and the k-th smallest weight will be distributed Beta(k, m)
- Like before, we can heuristically approximate the complexity as $\sum_{k=1}^{n-1} \frac{k}{m+1} = \frac{\Theta(n^2)}{\Theta(n \log n)} = \Theta(\frac{n}{\log n})$.

Dimension 2 - Unit Square

Claim: Expected total MST weight complexity is O($n^{1/2}$).

Heuristical Justification:

- Points are generated uniformly in a unit square
- Suppose, given n vertices, that we partition our square into n parts (see diagram below)



- 1) There will be $\approx \sqrt{n}$ squares along each side (since we're partitioning into n squares total)
- 2) Since there are n squares and we're uniformly generating n vertices in this unit square, on average, points will deviate from the closest other point by around "1 square side length"
- 3) Since there are $\sqrt{n}$ squares per side, and each side has length 1, this suggests that asymptotically, the average distance between one point and the closest point to that point is on the order of $\frac{1}{\sqrt{n}}$

- Since a MST on n vertices contains n-1 edges, we get a crude, purely heuristical estimate of $\Theta(n \cdot \frac{1}{\sqrt{n}}) = \Theta(n^{1/2})$ for the asymptotic complexity of the expected total MST weight as a function of n

Dimension 3 - Unit Cube
Claim: Expected total MST weight complexity is $O(n^{2/3})$.
Heuristical Justification:

- We generalize our dim 2 approach
- In dim 3, points are generated uniformly in a unit cube
- Suppose, given n vertices, that we partition our cube into n cubes (no diagram anymore, don't know how to do 3 dimensions)
  - 1) There will be $\approx n^{1/3}$ squares along each edge (since we're partitioning into n cubes total)
  - 2) Since there are n cubes and we're uniformly generating n vertices in this unit cube, on average, points will deviate from the closest other point by around "1 cube side length"
  - 3) Since there are $n^{1/3}$ cubes per edge, and each edge has length 1, this suggests that asymptotically, the average distance between one point and the closest point to that point is on the order of $n^{-1/3}$

- Since a MST on n vertices contains n-1 edges, we get a crude, purely heuristical estimate of $\Theta(n \cdot n^{-1/3}) = \Theta(n^{2/3})$ for the asymptotic complexity of the expected total MST weight as a function of n

Dimension 4 - Unit Hypercube
Claim: Expected total MST weight complexity is $O(n^{3/4})$.

Heuristical Justification:
- Again, we generalize our approach from the other 2 "Euclidean cases"
- In dim 4, points are generated uniformly in a unit hypercube (4D)
- Suppose, given n vertices, that we partition our hypercube into n cubes (no diagram anymore, 4D is a little tough)
    - 1) There will be $\approx n^{1/4}$ squares along each (hyper)edge (since we're partitioning into n hypercubes total)
    - 2) Since there are n hypercubes and we're uniformly generating n vertices in this unit hypercube, on average, points will deviate from the closest other point by around "1 hypercube side length"
    - 3) Since there are $n^{1/4}$ cubes per (hyper)edge, and each (hyper)edge has length 1, this suggests that asymptotically, the average distance between one point and the closest point to that point is on the order of $n^{-1/4}$
- Since a MST on n vertices contains n-1 edges, we get a crude, purely heuristical estimate of $\Theta(n \cdot n^{-1/4}) = \Theta(n^{3/4})$ for the asymptotic complexity of the expected total MST weight as a function of n

- **Experiences with the random number generator**

The challenge with the random number generator was balancing two competing factors: 1) the desire to get reproducible results and 2) the need to get independent random number generators. Ultimately, we went with a method of using pseudo-random generators because while reproducibility is nice, independent trials are critical to producing meaningful data: if our rng isn't independent, our trials and potentially rng usages within our trials will be heavily correlated, leading to unreliable, high-variance results.

Therefore, we decided to use a pseudo random number generator called the Mersenne Twister, as opposed to using the same seed (or even switching between a few seeds) per trial as the latter option could induce correlations in our rng process if we weren't careful (and there are so many random trials we need to run that even switching seeds might get impractical). Independent trials were important because we wanted to lower-variance estimates of the average total MST weight

at each n-value (so that we could get a better curve fit for the average total MST weight as a function of n for each dimension).